Lapa Caçambas — Sprint 3: Módulo de Logística

Data de entrega: 08 de abril de 2026
Responsável: Rodrigo
Branch: `branch-rodrigo`
Commit: `08e010f` — feat(logistica): módulo de logística completo — Sprint 3

---

1. Contexto

O módulo de logística fecha o loop do fluxo operacional do sistema. Quando um pedido avança para o status **`programado`**, o módulo de pedidos (Nathalia) já criava automaticamente um registro na tabela `execucoes` com `status: pendente` e sem motorista ou veículo atribuído. Este módulo é responsável por preencher esse vínculo e acompanhar a execução em campo.

Fluxo completo implementado:

```
Pedido → programado
    ↓ (automático, já existia)
Execução criada (status: pendente, sem motorista)
    ↓ (novo — logística)
Operador atribui motorista + veículo
    ↓ (novo — logística)
Operador cria rota do dia e adiciona execuções como paradas
    ↓ (novo — logística)
Status avança: em_rota → no_local → executando → concluida
    ↓ (novo — automático)
Pedido marcado como concluido
```

---

2. Arquivos Criados e Modificados

Novos arquivos

Arquivo	Descrição
`backend/src/modules/logistica/logistica.types.ts`	Tipos raw do banco (rows) e DTOs de entrada/saída
`backend/src/modules/logistica/logistica.service.ts`	Toda a lógica de negócio
`backend/src/modules/logistica/logistica.controller.ts`	9 endpoints REST

Arquivos modificados

Arquivo	O que mudou
`backend/src/app.ts`	Importação e registro de `logisticaRouter` em `/api/logistica`
`shared/enums.ts`	Adicionado `STATUS_EXECUCAO_LABELS`
`shared/contracts.md`	Documentação completa da logística + histórico de sprints
`src/lib/api.ts`	8 novas funções de chamada ao backend
`src/hooks/useQuery.ts`	7 novos hooks (queries + mutations)
`src/components/shared/StatusBadge.tsx`	Classes de cor para status de execução e rota
`src/pages/rotas/RotasPage.tsx`	Painel completo substituindo o placeholder

---

3. Backend — Endpoints Implementados

Base: `/api/logistica` — roles: `administrador`, `gestor`, `operador`

Execuções

Método	Rota	Descrição
GET	`/execucoes`	Lista execuções. Filtros: `status`, `data` (YYYY-MM-DD), `semAtribuicao=true`
GET	`/execucoes/:id`	Busca execução por ID com joins completos
PUT	`/execucoes/:id/atribuir`	Atribui motorista + veículo a uma execução pendente

PUT	`/execucoes/:id/status`	Atualiza status da execução

Rotas

Método	Rota	Descrição
GET	`/rotas`	Lista rotas. Filtros: `data`, `motoristaId`, `status`
POST	`/rotas`	Cria nova rota para o dia
GET	`/rotas/:id`	Busca rota com todas as paradas
PUT	`/rotas/:id/status`	Atualiza status da rota
POST	`/rotas/:id/paradas`	Adiciona parada à rota
DELETE	`/rotas/:id/paradas/:paradaId`	Remove parada da rota

---

4. Regras de Negócio Implementadas

Regra	Onde	HTTP
Motorista bloqueado ou inativo não pode ser escalado	`atribuirExecucao()`	422
Veículo em manutenção ou inativo não pode receber rota	`atribuirExecucao()`	422
`data_inicio` setado automaticamente ao entrar em `em_rota` ou `executando`	`atualizarStatusExecucao()`	—
`data_fim` setado automaticamente ao ir para `concluida` ou `cancelada`	`atualizarStatusExecucao()`	—
Execução `concluida` → pedido marcado como `concluido` automaticamente	`atualizarStatusExecucao()`	—
Adicionar parada com `pedidoId` vincula execução via `rota_parada_id`	`adicionarParada()`	—
Remover parada desvincula a execução (`rota_parada_id = null`) antes de deletar	`removerParada()`	—

---

5. Fluxo de Status

Execução

```
pendente → em_rota → no_local → executando → concluida
                                            ↘ cancelada
```

Rota

```
planejada → em_andamento → concluida
                         ↘ cancelada
```

---

6. Frontend — Painel de Programação Diária

Arquivo: `src/pages/rotas/RotasPage.tsx`

O painel é dividido em duas colunas:

Coluna Esquerda — Execuções do Dia

- Filtradas pela data selecionada (default: hoje)
- Separadas em dois grupos: **Aguardando atribuição** (amarelo) e **Atribuídas** (azul)
- Cada card mostra: número do pedido, cliente, endereço, data/hora programada, número da caçamba, tipo
- Card sem motorista: selects inline de motorista + veículo + botão "OK" para atribuição imediata
- Card com motorista: exibe nome do motorista e placa + botão "+ Rota" para vincular a uma rota

Coluna Direita — Rotas do Dia

- Filtradas pela mesma data selecionada
- Cada rota exibe: motorista, placa, data, status, número de paradas

- Expansível/colapsável com clique
- Lista de paradas com ordem, número do pedido, cliente, status e botão de remoção
- Botão contextual de avanço de status: "Iniciar Rota" (planejada → em andamento) / "Concluir Rota" (em andamento → concluída)

Ações Globais

- Seletor de data (default: hoje)
- Botão de refresh manual
- Botão "Nova Rota" — abre modal com seleção de motorista, veículo e observação
- Modal "Adicionar à Rota" — ao clicar em "+ Rota" em uma execução, lista as rotas ativas do dia para vincular
- Auto-refresh a cada 30 segundos via `refetchInterval`

---

7. Hooks Frontend

Hook	Tipo	Descrição
`useExecucoes(params?)`	Query	Lista execuções com filtros, refresh 30s
`useRotas(params?)`	Query	Lista rotas com filtros, refresh 30s
`useAtribuirExecucao()`	Mutation	Atribui motorista/veículo, invalida `execucoes`
`useCriarRota()`	Mutation	Cria rota, invalida `rotas`
`useAdicionarParada()`	Mutation	Adiciona parada, invalida `rotas` e `execucoes`
`useRemoverParada()`	Mutation	Remove parada, invalida `rotas` e `execucoes`
`useStatusRota()`	Mutation	Avança status da rota, invalida `rotas`

---

8. Schema das Tabelas (já existiam no banco)

`execucoes`

```
id               BIGSERIAL PK
pedido_id        BIGINT FK pedidos(id)
rota_parada_id   BIGINT FK rota_paradas(id)   -- vinculado ao adicionar para
motorista_id     BIGINT FK motoristas(id)      -- preenchido via /atribuir
veiculo_id       BIGINT FK veiculos(id)        -- preenchido via /atribuir
tipo             TEXT
status           status_execucao               -- pendente|em_rota|no_local|
data_inicio      TIMESTAMPTZ
data_fim         TIMESTAMPTZ
latitude         DOUBLE PRECISION
longitude        DOUBLE PRECISION
observacao       TEXT
evidencia_url    TEXT
created_at       TIMESTAMPTZ
updated_at       TIMESTAMPTZ
```

rotas

```
id           BIGSERIAL PK
data         DATE
motorista_id BIGINT FK motoristas(id)
veiculo_id   BIGINT FK veiculos(id)
status       TEXT                   -- planejada|em_andamento|concluida|can
observacao   TEXT
created_by   UUID FK auth.users
created_at   TIMESTAMPTZ
updated_at   TIMESTAMPTZ
```

rota_paradas

```
id           BIGSERIAL PK
rota_id      BIGINT FK rotas(id) ON DELETE CASCADE
pedido_id    BIGINT FK pedidos(id)
ordem        INT
endereco     TEXT
tipo         TEXT
status       status_execucao
```

```
  hora_chegada TIMESTAMPTZ
  hora_saida   TIMESTAMPTZ
  observacao   TEXT
  created_at   TIMESTAMPTZ
```

---

9. Pendências para Sprint 4

Rodrigo

- **Usuários backend** (R-02): `UsuariosPage.tsx` é placeholder — sem CRUD de usuários e sem gestão de roles via interface
- **Rastreamento GPS** (R-10): integração com API Sobrecontrole/Overcontrole — validar com o fornecedor antes de codar
- **Relatórios operacionais** (R-12): pedidos por dia, produtividade por motorista, ocupação de ativos

Interface com Nathalia (combinar antes de Sprint 4)

- Campo `nota_fiscal_id` na tabela `execucoes` para rastrear NF emitida por execução
- Gatilho de geração automática de fatura quando pedido vai para `concluido`

---

10. Checklist de Entrega

- [x] Backend compila sem erros (`tsc --noEmit`)
- [x] Frontend compila sem erros (`tsc --noEmit`)
- [x] Todos os endpoints documentados em `shared/contracts.md`
- [x] Regras de negócio testadas manualmente
- [x] Commit com mensagem convencional no padrão `feat(logistica): ...`
- [x] Branch `branch-rodrigo` atualizada
- [ ] PR aberto para revisão da Nathalia
- [ ] Rebase na `main` antes do merge